

Zero to Hero Study Handbook: lora-qlora-finetuning-lab

This handbook is a static, file-grounded guide to help a new learner understand this repository from first principles and then work productively with it.

Module 1: Foundations & Architecture

What this project does

This project is a notebook-first NLP fine-tuning lab that compares two parameter-efficient tuning methods for emotion classification framed as text generation:

- LoRA on `distilgpt2`
- QLoRA (4-bit quantized base model + LoRA adapters) on `facebook/opt-350m`

The task is based on the Hugging Face dataset `dair-ai/emotion`, where each input text is mapped to one label from:

- `sadness`
- `joy`
- `love`
- `anger`
- `fear`
- `surprise`

The repository provides:

- A CLI for end-to-end runs (`run-all`) and app serving (`serve-app`)
- A core Python pipeline (`src/lora_qlora_lab/pipeline.py`) that runs data prep -> baseline evaluation -> training -> tuned evaluation -> reporting
- A Streamlit dashboard over generated artifacts
- Tutorial notebooks that inspect generated artifacts
- Tests for core utility behavior

Main use cases

- Learn PEFT workflow structure end-to-end on a real repo.
- Compare LoRA vs QLoRA outputs, runtime, and metrics under one protocol.
- Re-run experiments with modified sampling/training settings via `.env`.
- Inspect artifacts in notebooks and the Streamlit app.

Core paradigms and patterns used here

1. Pipeline orchestration (imperative workflow): One coordinator function, `run_pipeline(settings)`, executes all major stages in sequence.

2. Config-driven runtime: `Settings` (Pydantic `BaseSettings`) centralizes paths, model names, sample sizes, and training controls.
3. Parameter-efficient transfer learning: Adapter-based tuning (`peft`) instead of full model fine-tuning.
4. Prompt-as-classification pattern: Classification is cast as generation with deterministic prompts, then post-processed into labels.
5. Artifact-first analytics: CSV/JSON/HTML/Markdown outputs are persisted and then consumed by notebooks/app.
6. Functional utility style with lightweight structured models: Most logic is functional; `EvalRecord` dataclass is used for typed evaluation records.

Architecture and component interaction

```

User/Automation
|
| CLI: lora-qlora-lab run-all    OR    scripts/run_pipeline.py
v
src/lora_qlora_lab/cli.py
|
v
run_pipeline(settings) [src/lora_qlora_lab/pipeline.py]
|
+--> load_emotion_splits() + sample_splits() + save_raw_splits()
|     [data/emotion_dataset.py]
|
+--> build_eval_records() for baseline/tuned
|
+--> evaluate_model() baseline LoRA
|
+--> train_lora()
|     [training/fine_tune.py]
|
+--> evaluate_model() tuned LoRA
|
+--> evaluate_model() baseline QLoRA (4-bit)
|
+--> train_qlora()
|
+--> evaluate_model() tuned QLoRA
|
+--> save_predictions(), save_metrics(), save_charts(),
|     render_markdown_report(), save_metrics_json()
|     [eval/inference.py + reporting/reporting.py]
|
v
artifacts/ + models/

```

```
|
+--> notebooks/*.ipynb (analysis/tutorial)
+--> app/streamlit_app.py (dashboard)
```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Package metadata, dependencies, CLI registration	project.scripts: lora-qlora-lab = lora_qlora_lab.cli	Python >=3.12.10, <3.13, runtime deps transformers, peft, bitsandbytes, torch, etc.), pytest config LORA_MODEL_NAME, QLORA_MODEL_NAME, DATASET_NAME, TRAIN_SAMPLES, VALIDATION_SAMPLES, TEST_SAMPLES, BASELINE_EVAL_SAMPLES, TUNED_EVAL_SAMPLES, MAX_LENGTH, LORA_MAX_STEPS, QLORA_MAX_STEPS, SEED, HF_TOKEN
.env.example	Environment/config template	N/A	Canonical commands for setup, pipeline, notebooks, app dataset_name, model names, sample sizes, step counts, seed, hf_token
README.md	Human-facing project overview and command quickstart	N/A	CLI commands: run-all, serve-app; Streamlit port default 8502
src/lora_qlora_lab/config.py	Configuration settings model and path resolution	Settings, get_settings, ensure_directories	
src/lora_qlora_lab/cli.py	CLI entrypoints	run_all_cmd, serve_app_cmd, callback	

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/lora_qlora_lab/ src/lora_qlora_lab/ pipeline.py	src/lora_qlora_lab/ pipeline orchestration	src/lora_qlora_lab/ run_pipeline, _save_json	Calls all major modules; summary payload shape
src/lora_qlora_lab/ src/lora_qlora_lab/ data/emotion_data.py	src/lora_qlora_lab/ prompt construction, split sampling, tokenization	src/lora_qlora_lab/ EasyDict, LABELS, build_prompt, build_train_text, extract_label, load_emotion_splits, sample_splits, tokenize_for_causal_lm	Label order defines ID->label decoding; deterministic shuffle via seed
src/lora_qlora_lab/ src/lora_qlora_lab/ training/fine_tune.py	src/lora_qlora_lab/ QLoRA training routines	src/lora_qlora_lab/ train_qlora, _build_lora_config, _target_modules_for_model	LoRA hyperparams $\alpha=16$, $\alpha=32$, dropout 0.05), QLoRA 4-bit config (nf4, double quant)
src/lora_qlora_lab/ src/lora_qlora_lab/ eval/inference.py	src/lora_qlora_lab/ generation-based evaluation, metric computation	src/lora_qlora_lab/ evaluate_model, _load_model_and_tokenizer, save_predictions, eval_records_to_frame	max_new_tokens=4, $\text{do_sample}=\text{False}$, sklearn accuracy macro_f1
src/lora_qlora_lab/ src/lora_qlora_lab/ reporting/reporting.py	src/lora_qlora_lab/ calculation, chart/report generation, JSON-safe writing	src/lora_qlora_lab/ compute_deltas, save_metrics, save_metrics_json, save_charts, render_markdown_report	Jinja2 markdown template, Plotly HTML charts, NaN
src/lora_qlora_lab/ src/lora_qlora_lab/ logging_utils.py	src/lora_qlora_lab/ configuration	src/lora_qlora_lab/ configure_logging	report sanitization via _sanitize_for_json Loguru format + level INFO
scripts/run_pipeline_cli.py	CLI convenience script for full pipeline run	top-level script body	Uses configure_logging(), get_settings(), run_pipeline()
scripts/execute_notebook.py	Sequential notebook execution helper	execute_notebook, main, NOTEBOOKS	Notebook order and nbclient timeout (1800)
app/streamlit_app.py	Visualization/dashboard for saved artifacts	top-level Streamlit app code	Reads artifacts/metrics/summary.json, evaluation_metrics.csv, predictions.csv

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
tests/test_prompts.py	Validates prompt/text-label helpers	test_prompt_and_text, test_extract_label	Failure text, Label:" prompt suffix and unknown-label behavior
tests/test_sampling.py	Validates split sampling limits	test_sample_splits	Check request limits sample caps per split
tests/test_report.py	Validates metrics delta + JSON sanitization	test_compute_delta, test_save_metrics	Confirms gain is monotonic, normalizes NaN->null conversion
data/raw/*.json	Persisted sampled dataset splits	N/A	Row shape is {"text": str, "label": int}
models/lora_distill and models/qlora_opt350 artifacts/	Trained adapter outputs + Sampling metrics Evaluation/reporting outputs consumed by tutorials/app	N/A	adapter/adapter_model.safetensors, adapter_config.json, train_metrics.json, metrics/summary.json, tables/predictions.csv, charts/*.html, reports/lora_qlora_report.md
notebooks/*.ipynb	Tutorial-style artifact inspection notebooks	N/A	Ordered narrative from data/baselines to final comparison

Module 3: Core Execution Flows

Flow A: End-to-end CLI execution (run-all)

Entrypoint:

- `uv run lora-qlora-lab run-all`
- maps to `run_all_cmd()` in `src/lora_qlora_lab/cli.py`

Step-by-step:

1. `run_all_cmd()` loads settings via `get_settings()`.
2. `run_pipeline(settings)` is called.
3. Pipeline sets seed (`transformers.set_seed(settings.seed)`) and creates directories (`settings.ensure_directories()`).
4. Dataset is loaded from Hugging Face (`load_emotion_splits(settings.dataset_name, settings.seed)`), sampled (`sample_splits(...)`), and persisted to `data/raw/emotion_{train,validation,test}.json`.

5. Two eval sets are prepared from sampled test split:
 - `baseline_records` with `settings.baseline_eval_samples`
 - `tuned_records` with `settings.tuned_eval_samples`
6. Baseline and tuned records are saved under `artifacts/tables/`.
7. Four evaluation/training branches run in order:
 - Baseline LoRA model evaluation
 - LoRA training -> tuned LoRA evaluation
 - Baseline QLoRA model evaluation
 - QLoRA training -> tuned QLoRA evaluation
8. Metrics/predictions are aggregated into DataFrames.
9. Artifacts are generated:
 - predictions CSV
 - metrics CSV
 - Plotly charts (HTML)
 - Markdown report
 - summary JSON + per-method train-metrics JSON
10. CLI prints `summary["deltas"]` JSON.

Core orchestration snippet (actual structure):

```
summary = {
    "dataset": settings.dataset_name,
    "seed": settings.seed,
    "lora_model": settings.lora_model_name,
    "qlora_model": settings.qlora_model_name,
    "results": results_df.to_dict(orient="records"),
    "deltas": compute_deltas(results_df),
    "lora_train_metrics": lora_train_metrics,
    "qlora_train_metrics": qlora_train_metrics,
}
```

Flow B: Data and prompt flow

Important functions:

- `build_prompt(text: str) -> str`
- `build_train_text(text: str, label: str) -> str`
- `decode_label(label_id: int) -> str`
- `extract_label(prediction: str) -> str`
- `tokenize_for_causal_lm(dataset, tokenizer, max_length) -> Dataset`

Prompt format used by training and evaluation:

Classify the emotion in the given text. Choose exactly one label from: sadness, joy, love, a
 Text: <original text>
 Label:

Label ID mapping comes directly from LABELS order in `emotion_dataset.py`:

- 0 -> sadness
- 1 -> joy
- 2 -> love
- 3 -> anger
- 4 -> fear
- 5 -> surprise

Input/output shapes in this flow:

- Raw sampled row (data/raw/*.json):
 - {"text": "<string>", "label": <int>}
- Evaluation record (EvalRecord dataclass):
 - text: str
 - gold_label: str
 - prompt: str

Flow C: LoRA training flow (train_lora)

Function:

- train_lora(settings, sampled_splits, output_dir) -> tuple[Path, dict[str, Any]]

Key operations:

1. Seed and output directory setup.
2. Tokenizer load for settings.lora_model_name.
3. Tokenize sampled train/validation splits with tokenize_for_causal_lm.
4. Load base CausalLM with dtype:
 - float16 if CUDA available
 - else float32
5. Build and attach LoRA adapter config:
 - r=16, lora_alpha=32, lora_dropout=0.05
 - GPT-like target modules: ["c_attn", "c_proj"]
6. Train with Trainer and TrainingArguments (max_steps=settings.lora_max_steps, batch size 8, grad accumulation 2).
7. Save adapter + tokenizer under models/lora_distilgpt2/adapter.
8. Save train_metrics.json, return (adapter_dir, metrics_dict).

Returned metrics dict shape:

```
{
  "train": {"train_runtime": 0.0, "...": "..."},
  "eval": {"eval_loss": 0.0, "...": "..."},
  "model": "distilgpt2",
  "method": "lora",
  "train_samples": 1800,
  "validation_samples": 300
}
```

Flow D: QLoRA training flow (train_qlora)

Function:

- `train_qlora(settings, sampled_splits, output_dir) -> tuple[Path, dict[str, Any]]`

What differs from LoRA flow:

1. Uses `settings.qlora_model_name` (facebook/opt-350m by default).
2. Applies 4-bit quantization config:
 - `load_in_4bit=True`
 - `bnb_4bit_quant_type="nf4"`
 - `bnb_4bit_use_double_quant=True`
 - `bnb_4bit_compute_dtype=torch.float16`
3. Calls `prepare_model_for_kbit_training(base_model)`.
4. Uses smaller per-device batch sizes with higher accumulation:
 - train batch size 2, eval batch size 2, grad accumulation 4
5. Optimizer is set to `paged_adamw_8bit`.
6. Saves adapter to `models/qlora_opt350m/adapter`.

Target modules for OPT-like model names come from `_target_modules_for_model`:

- `["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`

Flow E: Evaluation and post-processing flow

Function:

- `evaluate_model(model_name, records, hf_token, run_name, adapter_path=None, quantized_4bit=False)`

Step-by-step:

1. `_load_model_and_tokenizer(...)` loads tokenizer and base model.
2. Optional adapter injection with `PeftModel.from_pretrained` when `adapter_path` is provided.
3. For each `EvalRecord`, the model generates up to 4 tokens (`max_new_tokens=4, do_sample=False`).
4. Generated suffix is decoded as `raw_completion`.
5. `extract_label(raw_completion)` finds the first label substring from the label list; else returns "unknown".
6. Per-sample predictions are accumulated in a `DataFrame`.
7. Metrics computed with `sklearn`:
 - `accuracy_score`
 - `f1_score(..., average="macro", zero_division=0)`
8. Returns:
 - `metrics: dict[str, Any]`
 - `predictions_df: pd.DataFrame`

Prediction row shape (artifacts/tables/predictions.csv):

```
{
  "run_name": "tuned_lora",
  "text": "...",
  "gold_label": "sadness",
  "raw_completion": "sadness, joy,",
  "pred_label": "sadness"
}
```

Metrics row shape (artifacts/metrics/evaluation_metrics.csv):

```
{
  "run_name": "tuned_qlora",
  "model_name": "facebook/opt-350m",
  "quantized_4bit": true,
  "adapter_path": ".../models/qlora_opt350m/adapter",
  "n_samples": 80,
  "accuracy": 0.3625,
  "macro_f1": 0.1404781929592695
}
```

Flow F: Reporting and visualization flow

Important functions:

- `compute_deltas(results_df)`
- `save_metrics(results_df, path)`
- `save_charts(results_df, chart_dir)`
- `render_markdown_report(...)`
- `save_metrics_json(payload, path)`

Artifacts generated by this flow:

- `artifacts/metrics/evaluation_metrics.csv`
- `artifacts/metrics/summary.json`
- `artifacts/metrics/lora_train_metrics.json`
- `artifacts/metrics/qlora_train_metrics.json`
- `artifacts/charts/accuracy_by_run.html`
- `artifacts/charts/macro_f1_by_run.html`
- `artifacts/reports/lora_qlora_report.md`

Dashboard consumption path:

- `app/streamlit_app.py` reads `summary.json` + `evaluation_metrics.csv`
- Displays gain metrics and bar charts
- Optionally loads `predictions.csv` and provides run-level filtering

Flow G: Notebook tutorial path

Notebook order encoded in `scripts/execute_notebooks.py`:

1. `notebooks/01_data_and_baselines.ipynb`
2. `notebooks/02_lora_finetuning_tutorial.ipynb`
3. `notebooks/03_qlora_finetuning_tutorial.ipynb`
4. `notebooks/04_comparison_and_report.ipynb`

In the current repo state, notebooks primarily inspect already-produced artifacts rather than re-implementing training code inline.

Module 4: Setup & Run Guide

Prerequisites for a clean machine

- Linux/macOS shell
- Git
- uv
- Python 3.12.10 (project pin in `.python-version`)
- Optional GPU + CUDA for faster training/inference

Install and environment setup

```
git clone https://github.com/pypi-ahmad/lora-qlora-finetuning-lab.git
cd lora-qlora-finetuning-lab
uv python pin 3.12.10
uv sync --dev
cp .env.example .env
```

Required/optional environment variables (`.env`)

Core keys used by Settings:

- `LORA_MODEL_NAME` (default: `distilgpt2`)
- `QLORA_MODEL_NAME` (default: `facebook/opt-350m`)
- `DATASET_NAME` (default: `dair-ai/emotion`)
- `TRAIN_SAMPLES` (default: 1800)
- `VALIDATION_SAMPLES` (default: 300)
- `TEST_SAMPLES` (default: 300)
- `BASELINE_EVAL_SAMPLES` (default: 40)
- `TUNED_EVAL_SAMPLES` (default: 80)
- `MAX_LENGTH` (default: 192)
- `LORA_MAX_STEPS` (default: 30)
- `QLORA_MAX_STEPS` (default: 12)
- `SEED` (default: 42)
- `HF_TOKEN` (optional)

Typical command sequences

Run full pipeline:

```
uv run lora-qlora-lab run-all
```

Alternative convenience script:

```
uv run python scripts/run_pipeline.py
```

Run tutorial notebooks in order:

```
uv run python scripts/execute_notebooks.py
```

Run Streamlit dashboard:

```
uv run lora-qlora-lab serve-app --port 8502
```

Run tests:

```
uv run pytest
```

External services and data dependencies

- Hugging Face Hub access is required for:
 - Dataset loading: `dair-ai/emotion`
 - Base model loading (`distilgpt2`, `facebook/opt-350m`, or your overridden values)
- `HF_TOKEN` is optional for public assets, but needed for gated/private models.

Migrations/seeding

- No database migrations are defined in this repository.
- Data “seeding” is the deterministic sampling + saving of dataset splits to `data/raw/` via `save_raw_splits`.

Export this handbook as PDF

This repository does not include a built-in PDF task, but this markdown is compatible with Pandoc:

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Optional (table of contents):

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md --toc -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

Ordered study plan for a new learner

1. Start with `README.md` and `pyproject.toml` to understand purpose, stack, and entry commands.

2. Read `src/lora_qlora_lab/config.py` to understand all runtime knobs and path resolution.
3. Read `src/lora_qlora_lab/data/emotion_dataset.py` to learn label mapping, prompt format, and sampling/tokenization.
4. Read `src/lora_qlora_lab/training/fine_tune.py` to understand LoRA vs QLoRA setup differences.
5. Read `src/lora_qlora_lab/eval/inference.py` to understand generation-based evaluation and metric computation.
6. Read `src/lora_qlora_lab/reporting/reporting.py` for delta logic, chart/report generation, and JSON sanitization.
7. Read `src/lora_qlora_lab/pipeline.py` to connect all pieces into the end-to-end flow.
8. Read `src/lora_qlora_lab/cli.py`, `scripts/run_pipeline.py`, and `app/streamlit_app.py` for entrypoints and user interface.
9. Read `tests/` to see what correctness properties are currently enforced.
10. Finish with notebooks and artifacts to correlate code intent with produced outputs.

Practice exercises (with solution outlines)

1. Exercise: Trace the exact order of baseline and tuned evaluations in the pipeline. Solution outline: In `run_pipeline`, the order is baseline LoRA -> train LoRA -> tuned LoRA -> baseline QLoRA -> train QLoRA -> tuned QLoRA.
2. Exercise: Write the exact prompt template used for evaluation. Solution outline: Use `build_prompt` in `emotion_dataset.py`; it includes label list, `Text: <...>`, and ends with `Label:.`
3. Exercise: Explain how integer labels become text labels. Solution outline: `decode_label(label_id)` indexes `LABELS`; mapping follows list order `[sadness, joy, love, anger, fear, surprise]`.
4. Exercise: Identify all places where deterministic behavior is intentionally introduced. Solution outline: `set_seed(settings.seed)` in pipeline and training functions; `shuffle(seed=seed)` in `sample_splits`; deterministic generation via `do_sample=False`.
5. Exercise: Compare LoRA vs QLoRA training argument differences in this repo. Solution outline: LoRA uses batch size 8/8 and grad accumulation 2; QLoRA uses 2/2 with accumulation 4 and `optim="paged_adamw_8bit"` plus 4-bit quantization config.
6. Exercise: Describe the schema of one row in `predictions.csv` and explain how `pred_label` is chosen. Solution outline: Row has `run_name, text, gold_label, raw_completion, pred_label`; `pred_label` comes from first matching label substring in `raw_completion` else `"unknown"`.

7. Exercise: Explain how `summary.json` avoids invalid JSON values like NaN. Solution outline: `save_metrics_json` calls `_sanitize_for_json`, converting NaN and `pd.NA` to `None` before `json.dumps(..., allow_nan=False)`.
8. Exercise: Find one documentation/code inconsistency and explain it. Solution outline: `train_qlora` docstring says “Fine-tune with QLoRA on TinyLlama” while default model is `facebook/opt-350m` in `Settings`.
9. Exercise: Identify exactly which files the Streamlit app requires to render the dashboard. Solution outline: Hard requirement: `artifacts/metrics/summary.json` and `artifacts/metrics/evaluation_metrics.csv`; optional: `artifacts/tables/predictions.csv`.
10. Exercise: Explain which tests guard core behavior and which important behaviors are not directly tested. Solution outline: Tests cover prompt helper behavior, split-size capping, delta math, and NaN normalization; they do not directly test end-to-end training/evaluation execution.

Learner Verification Checklist

Use this checklist to confirm end-to-end understanding:

- Can you explain `run_pipeline()` stage-by-stage without opening the file?
- Can you describe how `LABELS`, `decode_label`, and `extract_label` work together?
- Can you explain the exact LoRA vs QLoRA differences implemented in `fine_tune.py`?
- Can you describe the input and output schemas of `evaluate_model()`?
- Can you name every artifact produced under `artifacts/` and which function writes it?
- Can you explain how `compute_deltas()` derives gains from baseline/tuned rows?
- Can you explain how the Streamlit app maps artifact files to UI widgets?
- Can you modify one `.env` setting and predict which module/function behavior changes?
- Can you point to the current test coverage boundaries and missing integration checks?